

Decision-Centric Anomaly Detection for Tender/Auction Event Streams under Review Budgets

A reproducible testbed + reference implementation

Author: Sarthak Sharma Last updated: 2025-12-18

NDA note and scope

This repository is a public testbed for a tender/auction anomaly detection pipeline delivered under NDA. The production data and exact rules cannot be shared, so the public artifact uses an anonymized login-like surrogate dataset plus a synthetic generator. The pipeline, evaluation, and engineering are unchanged. See `docs/DOMAIN_CONTEXT.md` for the field mapping.

Abstract

This repository is a compact, end-to-end anomaly detection testbed for tender/auction event streams (demonstrated on a login-like surrogate). The focus is decision-centric: given a fixed human review budget (e.g., top 1% of sessions), which sessions should be reviewed first, and how stable is that prioritization under time drift?

The artifact includes:

- A repeatable pipeline (load -> validate -> sessionize -> features -> baselines -> ensemble -> reports/bundle)
- Budget-based evaluation (precision/recall at budget) and a threshold transfer sanity check (train -> later window)
- A lightweight drift summary (PSI + KS)
- A small model bundle and a reference FastAPI scorer for inspection

Problem framing

In operational tender/auction monitoring, the downstream constraint is not a global threshold but human capacity: analysts can review K sessions per day, not everything above score 0.73.

This testbed treats anomaly detection as a ranking problem:

- Define a budget b in $(0,1)$ = fraction of sessions that can be reviewed.
- Flag the top $\text{ceil}(bN)$ sessions by anomaly score.
- Evaluate how many risky sessions land in that shortlist.

In the quickstart workflow, `failed_events` is used as a proxy risk signal (not ground truth). If you have a real label (fraud/abuse/incident), the evaluation code is set up so you can swap it in cleanly.

Data and sessionization

The pipeline expects a simple event schema (see `src/tad/features/sessionize.py`):

- Login Timestamp

- User ID
- IP Address
- Country
- Browser Name and Version
- Device Type
- Login Successful (0/1)

These fields are a surrogate for tender/auction events; see docs/DOMAIN_CONTEXT.md for the mapping.

Events are grouped into user-day sessions (one row per user per day), then summarized into numeric features:

- counts (events, failed_events, unique countries/devices/browsers/hours)
- rates (failed_rate, unique_country_rate, unique_device_rate, etc.)
- behavioral indicators (session duration, events_per_hour, failed_per_hour)

This keeps the demo dataset small while still making the event -> session feature transition explicit.

Baselines

The baseline suite is deliberately small and review-friendly:

- Robust Z-score (median/MAD)
- Isolation Forest
- One-Class SVM
- Local Outlier Factor (novelty mode)

All baselines expose a uniform `.score(X)` interface through lightweight wrappers (src/tad/models/baselines.py).

Ensemble and calibration

Two ensemble options are supported:

- `ensemble_rank`: rank-average across members (scale-free, simple).
- `ensemble_fisher` (default): calibrated combination via empirical p-values + Fisher aggregation.

Why `ensemble_fisher`:

- Maps each model's score into an empirical one-sided p-value using the train window score distribution
- Combines p-values via Fisher's method: $-2 * \sum \log(p_i)$

This yields a monotone surprise statistic that is less sensitive to arbitrary score scaling.

Implementation:

- `src/tad/models/ensemble.py`: `empirical_pvalues`, `fisher_stat`, `fisher_ensemble`
- Quickstart writes a `model_bundle/` with `calibration.json` (quantile-CDF sketch) so the API can reproduce the same calibrated aggregation without needing the full training set.

Evaluation

Budget metrics

For each budget b , the evaluation reports:

- `precision`: fraction of flagged sessions that are risky (proxy via `failed_events > 0`)
- `recall`: fraction of risky sessions captured by the shortlist
- counts for traceability (`flagged_sessions`, `risky_flagged`, `risky_total`)
- `lift`: precision divided by the base risky rate (how much better than random)
- confidence bounds for precision/recall (Wilson intervals, 95%)

This is implemented in `src/tad/eval/metrics.py` (`compute_budget_metrics`). The report also includes base risk rate and the area under the precision-vs-budget curve.

Threshold transfer sanity check

Rank quality can look good in-sample but degrade when thresholds are applied out-of-sample. Quickstart also computes thresholds on the train window and applies them to the test window:

- choose threshold t_b so that train flags match budget b
- apply t_b to later data and report precision/recall

Implemented as `threshold_eval` in `src/tad/eval/metrics.py`.

Risk curve (proxy label)

The run also summarizes proxy risk rate by score quantile. A monotone increasing curve suggests the ranker is ordering higher-risk sessions earlier, even without true labels.

Ensemble agreement

To sanity-check robustness, the report includes Jaccard overlap between the top-k sessions from `ensemble_fisher` and `ensemble_rank`.

Drift check

A small drift summary compares early vs late windows:

- PSI (Population Stability Index)
- KS statistic

Implemented in `src/tad/eval/drift.py`.

Results and ablations (synthetic)

The numbers below use the repo's synthetic generator and proxy labels. They are not claims of production performance, but they make the evaluation harness concrete.

Configs:

- experiments/configs/quickstart_synthetic.yaml

- experiments/configs/quickstart_synthetic_high_drift.yaml

Scenario	Ensemble	Budget	Precision	Recall	Rank	Ensemble	Scenario
Synthetic (low drift)	fisher	0.01	0.7895	0.0404	0.01	0.7632	0.0391
Synthetic (low drift)	rank	0.01	0.7632	0.0391	0.01	0.7632	0.0391
Synthetic (low drift)	fisher	0.05	0.5947	0.1523	0.05	0.5737	0.1469
Synthetic (low drift)	rank	0.05	0.5737	0.1469	0.05	0.5737	0.1469
Synthetic (high drift)	fisher	0.01	0.6543	0.0223	0.01	0.6543	0.0223
Synthetic (high drift)	rank	0.01	0.6543	0.0223	0.01	0.6543	0.0223
Synthetic (high drift)	fisher	0.05	0.4757	0.0809	0.05	0.4757	0.0809
Synthetic (high drift)	rank	0.05	0.4757	0.0809	0.05	0.4757	0.0809

Observations:

- The calibrated Fisher ensemble is modestly more precise than rank-averaging on the same stream.
- Under higher drift, precision drops at fixed budget, illustrating why budget thresholds should be monitored.
- Lift at 1% budget is about 4x over the base rate in the low-drift synthetic run.
- Top-k overlap between Fisher and rank ensembles is about 0.73-0.75 at 1-5% budgets (synthetic).

Assumptions and threats to validity

- Proxy labels: `failed_events` is a stand-in for real risk labels.
- Surrogate data: the public dataset is a structural proxy for tender/auction events.
- Stationarity: within-window stationarity is assumed for calibration and drift summaries.
- Randomness: synthetic results are deterministic for `random_state=13` but still synthetic.

Related work (short)

- Anomaly detection surveys (e.g., Chandola et al., 2009) for problem framing and evaluation taxonomy.
- Isolation Forest (Liu et al., 2008), LOF (Breunig et al., 2000), and One-Class SVM (Schoelkopf et al., 2001).
- Fisher's method for p-value aggregation (Fisher, 1925).
- Budgeted ranking metrics (`precision@k/recall@k`) from information retrieval.
- Drift checks via PSI and KS from credit risk monitoring and distribution shift testing.

Reproducibility and what to run

Quickstart

```
make install
make quickstart          # runs on sample data/Login_Data.xlsx
make quickstart-synth    # runs on synthetic generator (no private data)
```

```
make report                                # rebuilds docs/TECHNICAL_REPORT.pdf
```

Outputs include:

- report.md (human-readable summary)
- scores.csv, top_sessions.csv
- budget_metrics.json, budget_threshold_eval.json, drift.json
- risk_curve.json, ensemble_agreement.json
- plots under plots/
- model_bundle/ (inspectable artifacts + calibration)

Config-driven runs

```
tad run-config experiments/configs/quickstart_sample.yaml
tad run-config experiments/configs/quickstart_synthetic.yaml
tad run-config experiments/configs/quickstart_synthetic_high_drift.yaml
```

See docs/EXPERIMENTS.md for supported config keys (including ensemble_method and synthetic_cfg).

Limitations and extensions

- Labels: the demo uses failed_events as a proxy. Real labels are preferred for claims of detection quality.
- Feature set: intentionally compact. Adding richer behavioral features (geo/IP reputation, device fingerprints, etc.)

is straightforward: extend build_feature_frame.

- Session definition: user-day is simple. For other domains, use sliding windows, session timeout logic, or

graph-based grouping.

- Calibration: the quantile-CDF sketch is meant for deployment convenience; if you have enough data, storing full

score distributions or using conformal techniques can improve calibration.

Appendix: file map

- src/tad/quickstart.py: end-to-end orchestration
- src/tad/features/: schema validation, sessionization, feature matrix
- src/tad/models/: baselines + ensembling + bundle writer
- src/tad/eval/: budget metrics, threshold transfer, drift summary, risk diagnostics, report writer
- docs/: reviewer guide, domain context, experiments, architecture, this report